

FROM IDEA TO REALITY / APP USING AI / 2026

MatchPoint AI

A smart tennis motion trainer that turns an ordinary smartphone into a motion-tracked racket and provides real-time 3D practice, shot analysis, and coaching on a computer.

FINAL SUBMISSION PACKAGE

VIDEO	PASTE PUBLIC GOOGLE DRIVE VIDEO LINK HERE
GITHUB	https://github.com/yarin-kovalski/Assignment4_exe3
KANBAN	https://trello.com/b/892SCMYP/matchpoint-ai-final-project

Project summary

MatchPoint AI uses Expo DeviceMotion data from a player's phone, sends it through a Socket.IO broker, and maps the calibrated motion to a tennis racket in a Three.js court. Custom physics converts the swing into ball speed, direction, spin, bounce, target results, and live coaching feedback. The player can practice standard feeds, target cones, and configurable wind conditions, then finish the session and download a detailed performance report.

Submission note

This PDF already contains the GitHub link and final Kanban screenshot. Replace the highlighted video placeholder after uploading the final video to Google Drive and setting it to Anyone with the link - Viewer.

What the application does

Phone connection

Expo Go QR flow, LAN Socket.IO connection, permission handling, live connection state, and tennis-ready ghost-racket calibration.

Motion intelligence

Quaternion orientation, acceleration and angular motion processing, smoothing and spike rejection, forehand/backhand classification, swing-speed estimate, and contact timing.

3D tennis training

Detailed Three.js court, racket, felt ball and net; forehand/backhand feeds; neutral, fast-flat, heavy-topspin and random styles; repeat, alternate and single-shot play.

Custom physics

Fixed-step flight with gravity, drag, Magnus force, court bounce, net interaction, world bounds, swept racket collision, and physical first-bounce rulings.

Target cones

Deep, regular and short-shot layouts; falling/resetting 3D cones; points, hits, hit rate and streak reporting based on the observed first bounce.

Wind training

Off/light/medium/strong wind, eight directions, visible flow, procedural audio and report tracking. Wind modifies the live ball flight within playable bounds.

Live feedback

Court Vision, hit/miss, target accuracy, speed, timing, stroke side, shot style, and animated spin, racket-face, arc and follow-through gauges.

Session analysis

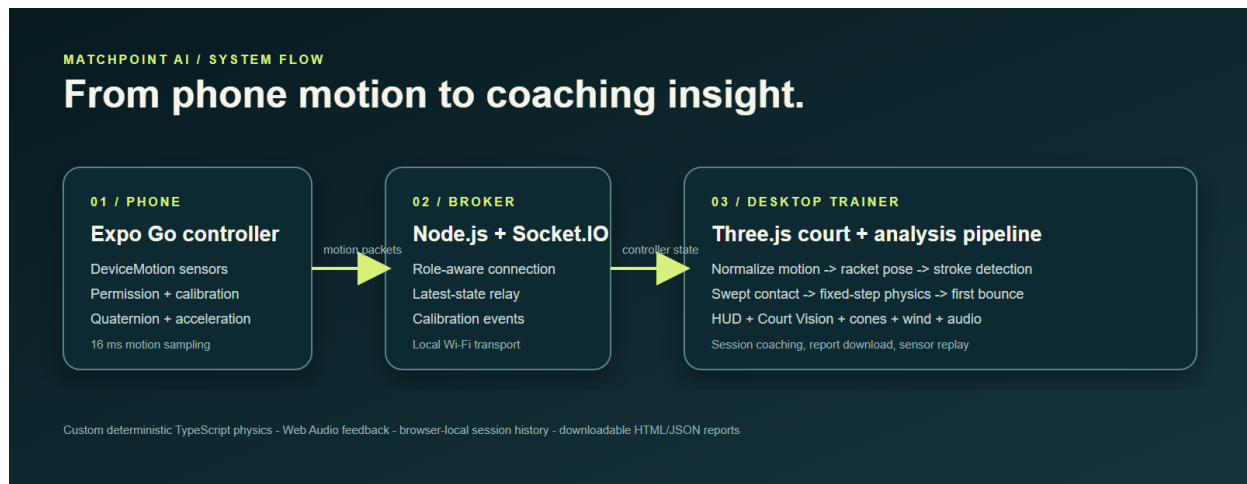
Player name and reflection, prior-session comparison, coaching priorities, cone/wind results, downloadable HTML report, and raw sensor JSON recording/replay.

Measurement limits

The system uses deterministic sensor processing and coaching rules rather than a trained neural network. Swing speed and technique levels are estimates from one phone. Sensor replay is not video replay, and the application does not claim body-pose or injury analysis.

Technology and system flow

LAYER	TECHNOLOGY	ROLE
Mobile	Expo SDK 57, React 19, React Native 0.86, expo-sensors	Phone UI and DeviceMotion sampling
Realtime	Socket.IO 4.7	Low-latency motion and calibration events
Server	Node.js, TypeScript	Serves clients, tracks roles and relays state
Desktop 3D	Three.js 0.184, WebGL, GLTFLoader	Court, racket, ball, lighting, targets and welcome scene
Physics	Project-owned TypeScript modules	Flight, spin, bounce, net, bounds and racket collision
Audio/data	Web Audio API, localStorage, HTML/JSON export	Physical sound cues, history and reports
Automation	PowerShell and npm scripts	LAN detection, build, Metro/server startup, QR and browser launch



Flow: the phone samples and calibrates DeviceMotion; Socket.IO carries compact packets through the Node.js broker; the PC validates and normalizes motion, updates the racket, evaluates the stroke/contact window, advances custom physics, rules the physical first bounce, and updates the HUD and session report.

Recommended video - 4:45 target

0:00-0:20	Opening	Cinematic welcome. Say the project name and one-line explanation.
0:20-0:45	Architecture	Show the diagram. Explain Expo sensors -> Socket.IO broker -> Three.js trainer, physics and feedback.
0:45-1:20	Connect	Show npm run all, QR scan, Connect, Start, live racket movement and ghost-racket calibration.
1:20-2:25	Training	Demonstrate forehand, backhand, feed styles, one-ball mode, Court Vision, timing, speed, audio and technique gauges.
2:25-3:20	Cone practice	Choose Deep Shot, hit a cone, show fall/reset, and briefly show Regular and Short Shot choices.
3:20-3:50	Wind	Enable wind, change strength/direction, show the flow and drift, and point out mute.
3:50-4:35	Report	Finish session; show identity/reflection, statistics, coaching, cone/wind results, and downloaded report.
4:35-4:55	Evidence	Show GitHub and Trello; mention automated sensor, physics, report, socket and browser checks.

Opening narration

This is MatchPoint AI, a smart tennis motion trainer that turns an ordinary smartphone into a motion-tracked racket and provides real-time 3D practice, shot analysis, and coaching on a computer.

Closing narration

MatchPoint AI makes advanced tennis practice accessible with equipment the player already owns: a phone and a computer.

Record at 1920x1080 or higher, export at 1080p, keep the final file below five minutes, demonstrate every claimed feature, and test the public Google Drive link in an incognito window.

Final Kanban board

MatchPoint AI — Final Project

MatchPoint AI is a smartphone-controlled tennis motion trainer built for the 2026 From Idea to Reality — App Using AI course. Expo and native phone sensors stream through a Node.js and Socket.io broker to a TypeScript and Three.js desktop trainer with calibrated racket motion, stroke analysis, and custom ball physics. Repository: https://github.com/yarin-kovalev/Assignment4_exe3

Backlog: 0

To Do: 3

In Progress: 0

Testing: 2

Done: 25

Cancelled / Parked: 4

Backlog

0

No cards

To Do

3

Record the final gameplay video

The repository documents a required final gameplay demonstration, but no commit proves that the final course video has been recorded. Record the phone controller, PC trainer, motion feedback, and the tested slice behavior in one clear demonstration.

Write the Moodle submission PDF

The graded submission still needs a PDF that explains the project and includes the video link, GitHub repository link, and final Kanban screenshot. No screenshot or PDF is claimed to exist yet.

Package the final source archive

A historical ZIP exists in the August startup commit, but the current Phase D.1 work is newer. A fresh archive must be generated from the approved final source rather than presenting the older ZIP as complete.

In Progress

0

No cards

Testing

2

Make feeds and racket contact playable

Start 2026-06-05 · Due 2026-09-17

Early feeds and exact collision rules made ordinary swings miss too often. This work moved feeds into natural reach, added bounded assistance near the real contact zone, tuned power for phone motion, and repeatedly refined direction and difficulty while preserving shot attributes.

History

- 1ac9e00 2026-08-05 fix(ball): prevent contact guide from crashing Three.js scene
- 2adff5a 2026-08-05 fix(ball): align readable one-bounce delivery with racket [BALL-01-READABILITY]
- dact1a5 2026-08-05 fix(ball): tune playable mirrored one-bounce deliveries [BALL-01-READABILITY]
- 660c734 2026-08-05 fix(ball): redesign easy feeds for reliable racket contact [BALL-01-READABILITY]
- 1c5a3d3 2026-08-05 fix(ball): add bounded easy-mode motion-assisted hits [BALL-01-READABILITY]
- a580e43 2026-08-05 fix(ball): move easy feeds closer along player's depth
- d5c40ef 2026-08-05 fix(ball): align easy feeds with stationary reach volume
- 0bba271 2026-08-05 fix(ball): move easy contact into natural arm reach
- 5580382 2026-08-06 fix(ball): make easy feeds enter the comfortable reach core
- 460709c 2026-08-06 feat(ball): add deterministic high-assist easy training feed
- 737ef67 2026-08-06 fix(diagnostics): expose and gate real hit recording controls
- 1a35589 2026-08-06 fix(ball): enforce forehand parity in calibrated play
- bee2304 2026-08-06 feat(ball): add safe varied player practice mode
- 2813c5 2026-08-07 perf(player): refine baseline positioning and feed rendering
- 8e0f618 2026-09-12 fix ball movement
- 036e6ba 2026-09-13 fix shot directions
- 380cd34 2026-09-13 improve shot
- 4208b70 2026-09-13 improve the racket shot
- 41086c6 2026-09-13 improve ball movement
- 0c77fcb 2026-09-13 IMPROVE BALL SPEED AND MOVEMENT
- 057941c 2026-09-13 set the shot direction and difficult level
- 3010a22 2026-09-14 fix difficult level of shots and success message
- b50e62d 2026-09-14 improve shot detection
- d6c732d 2026-09-16 fix shot difficult
- 6e11350 2026-09-16 improve shot speed
- 9540a50 2026-09-16 improve shot hardness
- 0622f42 2026-09-16 make the shot easier
- 0704f11 2026-09-17 improve player contact
- 9540011 2026-09-20 improve feed ball
- 94009f1 2026-09-20 move the court view slightly left by adjusting camera composition

Validate sensor-driven spin and slice trajectories

Start 2026-09-16 · Due 2026-09-17

Racket-face openness, brush direction, follow-through, topspin, and slice must produce distinct arcs and depths. Drop-slice and deep-slice profiles were calibrated from phone samples and the temporary calibration controls were removed; final real-phone acceptance is still required.

History

- f5532d8 2026-09-16 fix shot spin and high
- abc3b67 2026-09-16 improve open faces treatment
- 012d094 2026-09-16 verified trajectory behavior
- e530083 2026-09-17 calibrated the drop shot and deep slice
- 0b07719 2026-09-17 remove calibration slice and improve slice shot

Done

25

Invite the instructor to the repository

Start 2026-09-22 · Due 2026-09-22

Repository access is a submission action outside the code history. It must be completed and verified against the correct instructor account before the October 1 deadline.

Define the product and phased project plan

Start 2026-06-20 · Due 2026-06-20

The project needed a defensible problem, competitive context, architecture, risk register, and strict phase gate before implementation. The decision was to treat the tennis scene as a demonstration surface for motion analysis rather than build a game first.

History

- 706c979 2026-06-20 MatchPoint start

Build the phone-to-PC sensor pipeline

Start 2026-06-20 · Due 2026-06-20

The first technical risk was whether phone motion could reach the desktop reliably over the local network. This work established the TypeScript server, mobile sensor controls, calibration flow, desktop packet monitor, and HTTPS fallback notes before richer graphics were attempted.

History

- a1ec923 2026-06-20 Setting up the phone calibrate

Create the Three.js desktop proof of concept

Start 2026-06-20 · Due 2026-06-20

Raw packets were not enough for a convincing proof. The desktop was upgraded to a visible Three.js court and live phone-controlled object so latency and orientation behavior could be evaluated directly.

History

- f10f04e 2026-06-20 Add Three.js PC visualization

Expose a live swing-speed signal

Start 2026-06-20 · Due 2026-06-20

The Phase 1 acceptance criteria required a player-facing analytic signal rather than motion visualization alone. A conservative estimated swing-speed overlay was added to demonstrate that incoming sensor data could drive coaching-oriented metrics.

History

- 0189170 2026-06-20 Add Estimated swing speed

Replace the proxy with a centered racket model

Start 2026-06-20 · Due 2026-06-21

A generic 3D proxy could not communicate tennis orientation clearly. The GLB racket was introduced, centered, and repeatedly corrected around its local X axis until its head, grip, and face followed the phone in a believable orientation.

History

- 644ec0c 2026-06-20 Add racket visualization
- 9a0a1a1 2026-06-20 Add racket model centering

Cancelled / Parked

4

Defer MediaPipe body-pose tracking

Start 2026-06-20 · Due 2026-06-20

Body-pose tracking was documented as an optional future capability, but it was parked because the course prototype first needed reliable racket motion from one ordinary phone. No claim is made that player biomechanics beyond the tracked device are measured.

History

- 706c979 2026-06-20 MatchPoint start

Defer scored game mode and shooting targets

Start 2026-06-20 · Due 2026-06-20

The smart training analysis is now implemented, but scored game-mode targets and the larger arcade loop remain deliberately postponed until the trainer behavior and final course submission are verified.

History

- 706c979 2026-06-20 MatchPoint start

Reject a heavyweight physics engine for the prototype

Start 2026-08-05 · Due 2026-08-05

Cannon.js appeared in the early roadmap, but Phase D deliberately used small deterministic Three.js-compatible physics modules. This kept collision and replay testable and avoided adding engine complexity before scoring, opponents, or rally logic existed.

History

- 050e4dd 2026-08-05 feat: add 3D ball flight and racket collision prototype

Defer opponent, serving, scoring, and rally AI

Start 2026-08-05 · Due 2026-08-05

Phase D was explicitly constrained to one launch, one player contact, one return, and reset. Opponent behavior, serves, scoring, and rally state were parked so spatial collision and stroke timing could be verified independently.

History

- 050e4dd 2026-08-05 feat: add 3D ball flight and racket collision prototype

<https://trello.com/b/892SCMYP/matchpoint-ai-final-project>

MATCHPOINT AI / FINAL PROJECT 2026

5

Moodle upload checklist

- Push the final approved commits to the private GitHub repository.
- Confirm the instructor still has access to the repository.
- Record and export the final video at 1080p and five minutes or less.
- Upload the video to Google Drive and choose Anyone with the link - Viewer.
- Open the video link in an incognito browser and verify playback.
- Regenerate this PDF with the real video link.
- Upload MatchPoint-AI-Final-Submission.pdf to Moodle.
- Upload finalproject.zip to Moodle.
- Download both Moodle uploads and open them to confirm they are complete and not corrupted.

Regenerate the PDF after uploading the video

PowerShell:

```
$env:MATCHPOINT_VIDEO_URL='https://drive.google.com/your-public-video-link'  
python scripts/create_submission_pdf.py
```

Required Moodle files

MatchPoint-AI-Final-Submission.pdf	Video URL, GitHub URL, project information, architecture, and final Kanban screenshot.
finalproject.zip	Source code archive without Git history, dependencies, temporary output, credentials, or local caches.